


# 2 – Création de versions

Hadrien Grasland

IJCLab – Orsay

19/10/2020

# Notion de version

- **Il y a deux manières de voir une « version » :**
  - Un état sauvegardé d'une arborescence de fichiers
  - Des changements par rapport à une version précédente
- **Gardez bien à l'esprit ces deux points de vue**
  - Suivant la situation, l'un ou l'autre sera plus utile

# Façonnage d'une version

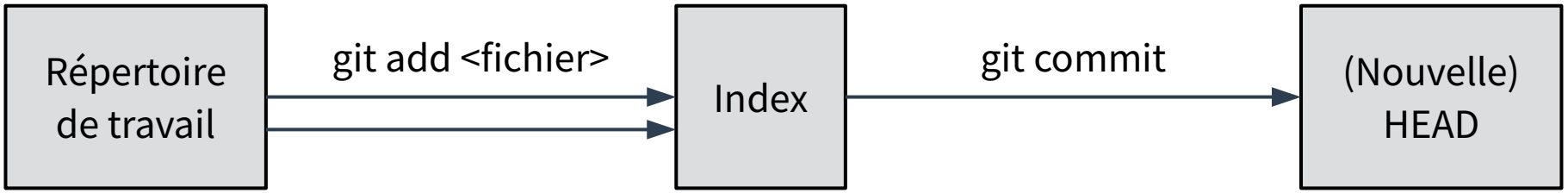
- **Git permet de créer une version progressivement**
  - On sélectionne quels changements seront sauvegardés
  - Ce processus de sélection est itératif
- **Pour cela, on manipule trois arborescences**
  - « Passé » : La version d'origine sur laquelle on se base
  - « Présent » : Le contenu actuel du répertoire de travail
  - « Futur » : La version que l'on est en train de créer

# Eléments de vocabulaire

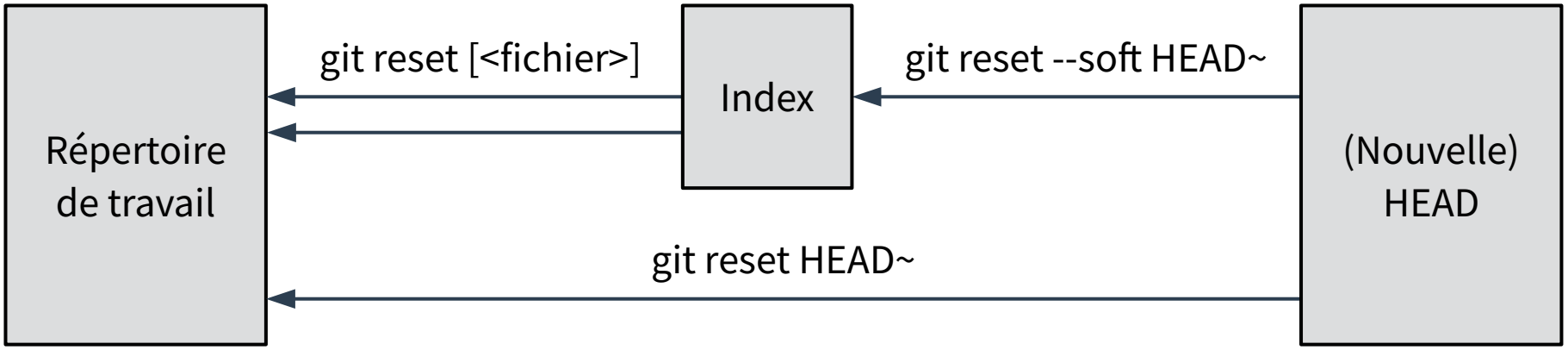
- **Mettons des mots sur tous ces concepts :**
  - Commit : Une version de notre projet / ses changements
  - HEAD : La version sur laquelle on base notre travail
  - Index, *staging area* : La version en cours de création
  - Répertoire de travail : Les fichiers que l'on manipule

# Processus de création

- En partant des changements du répertoire…




- …il est toujours possible de revenir en arrière



# Qu’est-ce qu’un bon commit ?

- **Un ensemble de changements cohérent**
  - Idéalement, chaque commit est auto-suffisant
  - Ex : Le code compile, les noms de fichiers sont bons…
- **Une ampleur réduite**
  - Commits fréquents = Retour en arrière facilité
  - Simplifie la localisation d’erreurs, la revue de code…
  - Mais on peut débattre de la bonne granularité

# Message de commit

- **« git commit » demande de saisir une description**
  - Sera attachée à la nouvelle version que l'on va créer
  - But : Résumer et expliquer les changements effectués
- **Quelques conseils d'écriture :**
  - Se positionner par rapport à la version précédente
  - Commencer par un résumé court (< 80 caractères)
  - Compléter par une description plus longue, si nécessaire

# Quelques styles de commits

**« Correctif pour le bug #123 »**

**« Élimine une perte de données dans les backups**

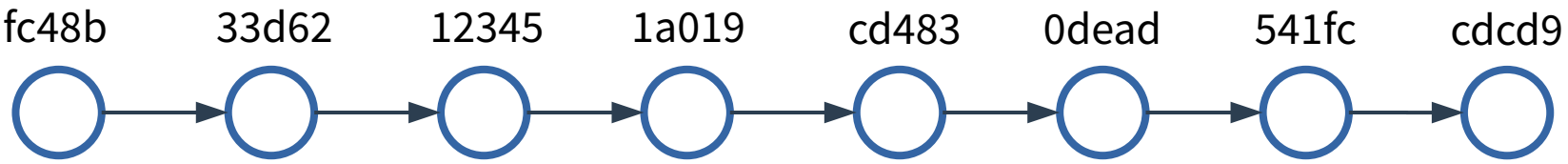
Évite le formatage accidentel de la partition système lorsque l'heure est incorrecte. »

**« Passage du module d'interface utilisateur en C++11**

- Pointeurs nus remplacés par des unique_ptr
- Support de la sémantique de mouvement
- Mesure de temps par std::chrono »

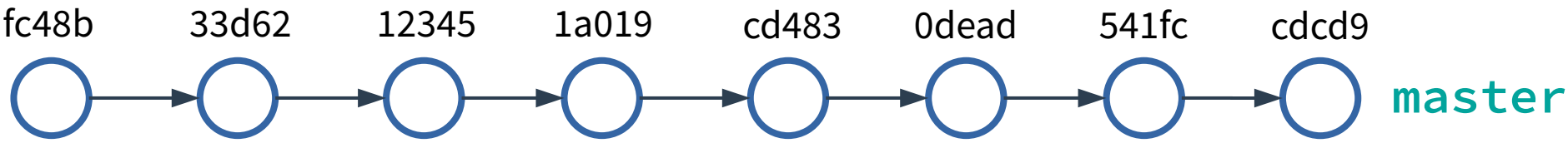
# Construction d’un historique

- Chaque commit est identifiable par un *hash*
- De commit en commit, on construit un historique



# Construction d'un historique

- **Chaque commit est identifiable par un *hash***
- **De commit en commit, on construit un historique**




- **Une branche = Une extrémité de l'historique**
  - Suffisant car chaque commit connaît ses parents
  - A chaque commit créé, la branche actuelle avance
  - Par défaut, il y a une branche appelée « master »

# Décortiquons « git status »

$ git status

Sur la branche master — Branche active

Validation initiale — Le premier commit est spécial (pas de parents)

Modifications qui seront validées :
(utilisez "git rm --cached <fichier>..." pour désindexer)

nouveau fichier : .gitignore
nouveau fichier : MonDossier/Contenu.txt

— Index (prochain commit)

Fichiers non suivis:
(utilisez "git add <fichier>..." pour inclure dans ce qui sera validé)

UnAutreFichier.txt

— Répertoire de travail

# Démo : Bases de gitg

# TP : Historique linéaire